

Let's rewrite `orderBurgers()` to use one positional parameter and three keyword parameters with default values, like this:

```
def orderBurgers(nBurgers, ketchup=True, mustard=True, pickles=True):
```

When we make a call to this function, `nBurgers` is a positional parameter and therefore must be specified as an argument in every call. The other three are keyword parameters. If no values are passed for `ketchup`, `mustard`, and `pickles`, the function will use the default value of `True` for each of those parameter variables. Now we can order two burgers with all the condiments like this:

```
orderBurgers(2)
```

If we want something other than a default value, we can specify the name of the keyword parameter and a different value in our call. For example, if we only want ketchup on our two burgers, we can make the call this way:

```
orderBurgers(2, mustard=False, pickles=False)
```

When the function runs, the values of the `mustard` and `pickles` variables are set to `False`. Since we did not specify a value for `ketchup`, it is given the default of `True`.

You can also make the call specifying all arguments positionally, including those written as keyword parameters. Python will use the ordering of your arguments to assign each parameter the correct value:

```
orderBurgers(2, True, False, False)
```

In this call, we are again specifying two burgers with ketchup, no mustard, and no pickles.

Additional Notes on Keyword Parameters

Let's quickly go over a few conventions and tips for using keyword parameters. As a Python convention, when you use keyword parameters and keywords with arguments, the equal sign between the keyword and the